

W13D03 — SQL DML

INSERT · UPDATE · DELETE · RETURNING | ML Engineer Journey Phase 4: Database & Observability | 3 Jun 2026

1. Posisi dalam Roadmap

W13D01 membangun koneksi ke PostgreSQL. W13D02 mendesain schema dengan DDL. Hari ini adalah kelanjutan langsung: schema yang ada diisi, diedit, dan dihapus isinya menggunakan DML. Semua constraints yang didefinisikan di DDL (FK, NOT NULL, UNIQUE) aktif dicek setiap kali DML dijalankan — defense in depth antara Pydantic di aplikasi dan PostgreSQL di database.

2. Analogi: Lab Notebook

Schema SQL = template halaman lab notebook: kolom, tipe data, aturan validasi. Halaman kosong, siap diisi. DML = aksi menulis di halaman itu. INSERT = menulis entri baru. UPDATE = mencoret nilai dan menulis ulang. DELETE = merobek satu baris. RETURNING = setelah menulis, database langsung berkata: 'ini id baris yang baru saja kamu buat' — tanpa perlu SELECT terpisah.

3. DDL vs DML

Aspek	DDL (W13D02)	DML (W13D03)
Singkatan	Data Definition Language	Data Manipulation Language
Fungsi	Mendefinisikan struktur	Memanipulasi isi
Contoh	CREATE TABLE, ALTER TABLE, DROP	INSERT, UPDATE, DELETE
Efek pada data	Tidak ada — hanya struktur	Mengubah isi tabel
Constraint	Mendefinisikan aturan	Menjalankan aturan

4. INSERT

INSERT menambahkan satu atau lebih baris ke tabel. RETURNING adalah fitur spesifik PostgreSQL yang mengembalikan data dari row yang baru diinsert — termasuk kolom yang di-generate DB seperti UUID atau TIMESTAMPTZ. Tanpa RETURNING kamu harus jalankan SELECT terpisah, yang rawan race condition di sistem concurrent.

SQL — INSERT + RETURNING

```
-- INSERT satu baris dengan RETURNING
INSERT INTO predictions
    (model_id, input_data, prediction, confidence, latency_ms)
VALUES
    (%s, %s, %s, %s, %s)
RETURNING id, created_at;
-- id (UUID) dan created_at (TIMESTAMPTZ) di-generate DB, dikembalikan langsung
```

Python — Bulk INSERT dengan execute_values()

```
from psycopg2.extras import execute_values, Json

sql = """
    INSERT INTO predictions (model_id, input_data, prediction, confidence, latency_ms)
    VALUES %s
    RETURNING id
    """
values = [
    (row["model_id"], Json(row["input"]), row["pred"], row["conf"], 45)
    for row in batch
]
execute_values(cur, sql, values)
ids = [str(r[0]) for r in cur.fetchall()]
# execute_values = satu round trip untuk banyak baris, lebih efisien dari executemany()
```

5. UPDATE

UPDATE mengubah nilai kolom pada baris yang memenuhi kondisi WHERE. Aturan paling penting: SELALU sertakan WHERE. UPDATE tanpa WHERE mengubah SEMUA baris di tabel.

PERINGATAN: UPDATE tanpa WHERE = seluruh tabel berubah. Tidak bisa di-undo tanpa transaction.

SQL — UPDATE patterns

```
-- AMAN: update spesifik
UPDATE predictions
SET confidence = %s
WHERE id = %s
RETURNING id, confidence;

-- BERBAHAYA: update semua baris
-- UPDATE predictions SET confidence = 0.91; <-- jangan pernah tanpa WHERE di production

-- Interval dinamis: lebih aman dari string formatting
UPDATE predictions
SET latency_ms = NULL
WHERE created_at < NOW() - (INTERVAL '1 day' * %s)
RETURNING id;
```

6. DELETE

Sama seperti UPDATE, SELALU sertakan WHERE. Bedakan tiga command:

Command	Efek	Rollback?	Kecepatan
---------	------	-----------	-----------

DELETE FROM t WHERE ...	Hapus rows tertentu	Ya (dalam transaction)	Normal
DELETE FROM t	Hapus semua rows, tabel tetap	Ya	Lambat
TRUNCATE TABLE t	Hapus semua rows	Terbatas	Cepat
DROP TABLE t	Hapus tabel + semua isinya	Tidak	Instan

SQL — DELETE patterns

```
-- DELETE satu baris, konfirmasi via RETURNING
DELETE FROM predictions
WHERE id = %s
RETURNING id;

-- DELETE berdasarkan kondisi (data retention job)
DELETE FROM predictions
WHERE created_at < NOW() - (INTERVAL '1 day' * %s)
AND confidence < 0.5
RETURNING id, prediction;
```

7. FK Constraint saat DELETE (ON DELETE RESTRICT)

ON DELETE RESTRICT — yang kamu define di W13D02 — aktif saat DELETE dijalankan. PostgreSQL menolak penghapusan parent row jika masih ada child row yang mereferensikannya. Ini bukan bug, ini fitur: database melindungi integritas referensial dari semua jalur masuk.

SQL — FK Constraint + solusi

```
-- Coba hapus model yang masih direferensikan predictions
DELETE FROM models WHERE id = 1;
-- ERROR: update or delete on table "models" violates foreign key constraint
-- DETAIL: Key (id)=(1) is still referenced from table "predictions".

-- Solusi 1: hard delete bertahap
DELETE FROM predictions WHERE model_id = 1; -- hapus child dulu
DELETE FROM models WHERE id = 1;             -- baru hapus parent

-- Solusi 2: soft delete
ALTER TABLE models ADD COLUMN IF NOT EXISTS deleted_at TIMESTAMPTZ;
UPDATE models SET deleted_at = NOW() WHERE id = 1;
-- Data tetap utuh untuk audit trail
```

Solusi	Cara	Kapan dipakai
Hard delete bertahap	Hapus child rows dulu, lalu parent	Data tidak perlu audit trail, relasi tidak relevan lagi
Soft delete	Tambah kolom deleted_at, UPDATE bukan DELETE	Production data, perlu audit trail, relasi masih relevan

8. JSON Serialization di psycopg2

psycopg2 tidak punya type mapping untuk Python dict. dict harus diubah ke format yang bisa dikirim ke PostgreSQL. Ada tiga cara, dua untuk arah Python ke DB, satu untuk arah DB ke Python:

Cara	Arah	Catatan
------	------	---------

json.dumps(data)	Python -> DB	Eksplisit, selalu aman, mudah debug
Json(data) dari psycopg2.extras	Python -> DB	Idiomatis psycopg2, handle True/None/False dengan benar
register_json(conn)	DB -> Python	Deserialization saja — data dari DB menjadi Python dict

Jebakan: pass Python dict mentah bisa bekerja untuk kasus sederhana, tapi gagal saat ada boolean atau null. Python True/False/None berbeda dari JSON true/false/null. Selalu gunakan Json() atau json.dumps().

Python — Json() dan register_json()

```
from psycopg2.extras import Json, register_json

# Setup saat connect – aktifkan auto-deserialization dari DB
conn = psycopg2.connect(**DB_CONFIG)
register_json(conn) # DB -> Python: fetchone() return dict, bukan string

# Saat INSERT – wrap dict dengan Json()
params = (model_id, Json(input_data), result, confidence, 45)
# Json() memberitahu psycopg2: serialize ini sebagai JSON ke DB
```

9. Pola Production-Grade psycopg2

Python — Complete production pattern

```
conn = None
try:
    conn = psycopg2.connect(**DB_CONFIG)
    with conn.cursor() as cur:
        cur.execute(sql, params)
        row = cur.fetchone() # None jika tidak ada hasil
    conn.commit()           # wajib – default autocommit=False

except psycopg2.Error as e:
    print(f"Database error: {e}")
    if conn:
        conn.rollback()     # batalkan semua perubahan dalam transaction aktif

finally:
    if conn:
        conn.close()        # SELALU tutup – cegah resource leak (max 100 connections!)

# Tuple satu elemen: (value,) – koma wajib, bukan (value)
cur.execute("DELETE FROM t WHERE id = %s", (prediction_id,))

# rowcount vs RETURNING:
# cur.rowcount = jumlah rows terpengaruh (integer metadata, tersedia tanpa RETURNING)
# RETURNING   = data riil dari rows – wajib jika butuh id/nilai/timestamp
```

Next — W13D04: SELECT, JOIN & Query Patterns

Schema ada (W13D02). Data bisa diisi dan dimanipulasi (W13D03). Berikutnya: baca data efisien dengan SELECT, gabungkan tabel dengan JOIN, dan baca EXPLAIN untuk memahami query plan — fondasi query optimization.

